# Linkers and Loaders

CS 348
Implementation of Programming Languages Lab
Computer Science and Engineering Department
Indian Institute of Technology
Guwahati

# Contents

# Overview

A linker is a system software tool that:

- Combines multiple object modules
- Resolves symbolic references between modules
- Produces a single executable file

Why do we need Linkers?

Modern Programs:

- Are modular
- Use libraries
- Are written across multiple files

Linkers are necessary to connect them all and execute the program.

# Need for Linker

**Example:**

Module A:

```
extern FUNC
```

Module B:

```
FUNC: function definition
```

Assembler cannot resolve address of FUNC across modules, linker solves this.

# Major Functions

- Address Assignment
- External Symbol Resolution
- Relocation
- Section Merging
- Library Handling

# Address Assignment

Assign final memory addresses to each module and section.

Each object file assumes base address = 0 (if not explicitly assigned)

Linker Action:

- Determine base address for each module.
- Assign sequential memory locations.

  Example:

| Module | Base |
|---|---|
| A | 1000 |
| B | 1400 |

# Section Merging

Each object file contains sections:

- `.text` (code)
- `.data` (initialized data)
- `.bss` (uninitialized data)

Linker mergers into:

```
Final .text = A.text + B.text
Final .data = A.data + B.data
Final .bss  = A.bss  + B.bss
```

# External Symbol Resolution

- Matches external references with definitions
- Builds External Symbol Table (ESTAB)

ESTAB (External Symbol Table) is a table maintained by the linker that stores:

- Symbol name
- Final absolute address
- Module in which it is defined

# Two Pass Linker

During Pass I:

- Read object file headers.
- Assign base addresses.
- Insert defined symbols into ESTAB.
- Detect duplicate symbol definitions.

If:

- Module A defines FUNC
- Module B defines FUNC
- Raise Linker error: Duplicate symbol

# Two Pass Linker

Pass II:

- Replace undefined symbol references
- Apply relocation entries
- Generate executable image

If:

- Module A defines FUNC
- Module B defines FUNC
- Raise Linker error: Duplicate symbol

# Relocation

Base address for each module is assigned at the address assignment phase.

After symbol resolution the addresses may change according to the base address value, re-computation and verification of such addresses is required.

Example:

```
Module B base = 1400
Instruction Offset = 20

Final Address: 1400 +20 = 1420
```

# Library Handling

When program uses libraries:

```
printf()
```

Linker:

- Searches library archive.
- Extracts only required object module.
- Adds its symbols to ESTAB.
- Performs relocation.

  Unused library modules are ignored.

# Full Linking Flow

Step 1: Assign addresses
Step 2: Build ESTAB
Step 3: Detect duplicate definitions
Step 4: Resolve external references
Step 5: Perform relocation
Step 6: Merge sections
Step 7: Produce final executable

# Example

Module A:

```
MAIN    START 0
        EXTREF  VAL
        LDA     VAL
        END
```

Object Program:

```
H^MAIN^000000^000003
R^VAL
T^000000^03^000000
M^000000^05^+VAL
E^000000
```

# Example

Module A:

```
MAIN    START 0
        EXTREF  VAL
        LDA     VAL
        END
```

Object Program:

```
H^MAIN^000000^000003
R^VAL
T^000000^03^000000
M^000000^05^+VAL
E^000000
```

- H → Header (name, start, length = 03 hex)
- R → External reference (VAL)
- T → Text record
- M → Modification record (needs VAL address added)
- E → End record

# Example

Module B:

```
SUB     START 0
VAL     WORD  7
        END
```

Object Program:

```
H^SUB^000000^000003
D^VAL^000000
T^000000^03^000007
E^000000
```

- H → Header (name, start, length = 03 hex)
- D → Defines symbol VAL at address 0 within SUB
- T → Text record
- E → End record

# Example

Assume linking starting address = 4000 (hex)

Object Program:

H^MAIN^000000^000003
R^VAL
T^000000^03^000000
M^000000^05^+VAL
E^000000

H^SUB^000000^000003
D^VAL^000000
T^000000^03^000007
E^000000

| Control Section | Length | Assigned Base |
| --- | --- | --- |
| | | |
| | | |

# Example

Assume linking starting address = 4000 (hex)

Object Program:

H^MAIN^000000^000003
R^VAL
T^000000^03^000000
M^000000^05^+VAL
E^000000

H^SUB^000000^000003
D^VAL^000000
T^000000^03^000007
E^000000

| Control Section | Length | Assigned Base |
|---|---|---|
| MAIN | 000003 | 4000 |
| | | |

# Example

Assume linking starting address = 4000 (hex)

Object Program:

H^MAIN^000000^000003
R^VAL
T^000000^03^000000
M^000000^05^+VAL
E^000000

H^SUB^000000^000003
D^VAL^000000
T^000000^03^000007
E^000000

| Control Section | Length | Assigned Base |
|---|---|---|
| MAIN | 000003 | 4000 |
| SUB | 000003 | 4003 |

# Example

Assume linking starting address = 4000 (hex)

Object Program:

H^MAIN^000000^000003
R^VAL
T^000000^03^000000
M^000000^05^+VAL
E^000000

H^SUB^000000^000003
D^VAL^000000
T^000000^03^000007
E^000000

**External Symbol Table**

| Symbol | Address |
|---|---|
| MAIN | 4000 |
| SUB | 4003 |
| | |

# Example

Assume linking starting address = 4000 (hex)

Object Program:

H^MAIN^000000^000003
R^VAL
T^000000^03^000000
M^000000^05^+VAL
E^000000

H^SUB^000000^000003
D^VAL^000000
T^000000^03^000007
E^000000

**External Symbol Table**

| Symbol | Address |
|---|---|
| MAIN | 4000 |
| SUB | 4003 |
| VAL | 4003 |

# Example

Now linker processes:

Step 1: Load Text Records

MAIN: T^000000^03^000000
SUB:  T^000000^03^000007

Step 2: Process Modification Record

PROGA: M^000000^05^+VAL

# Example

Final Executable:

H^PROG^004000^000006

T^004000^03^004003

T^004003^03^000007

E^004000

# Linker Guarantees

Before producing executable:

- No unresolved symbols
- No duplicate definitions
- All relocations applied
- Memory layout finalized

# Types of Linkers

Static Linker:

- All library code is copied into executable.
- No runtime dependency.

Dynamic Linker:

Libraries are linked at runtime.

Executable contains references to shared libraries.

# Static Linker

Steps:

- Identify required library functions
- Copy relevant object modules
- Merge sections
- Resolve symbols
- Produce standalone executable

# Static Linker

Advantages:

- Self-contained executable
- Faster startup
- No external dependencies

Disadvantages:

- Large executable size
- Code duplication across processes
- Library updates require relinking

# Dynamic Linker

Steps:

Build time:

- Record unresolved external references

Runtime:

- Dynamic linker resolves symbols
- Shared libraries loaded into memory

# Dynamic Linker

Advantages:

- Smaller executables
- Shared memory usage
- Library updates without recompilation

Disadvantages:

- Runtime overhead
- Version conflicts
- Dependency management complexity

# ELF FORMAT

ELF = Executable and Linkable Format

Used in Linux and Unix systems.

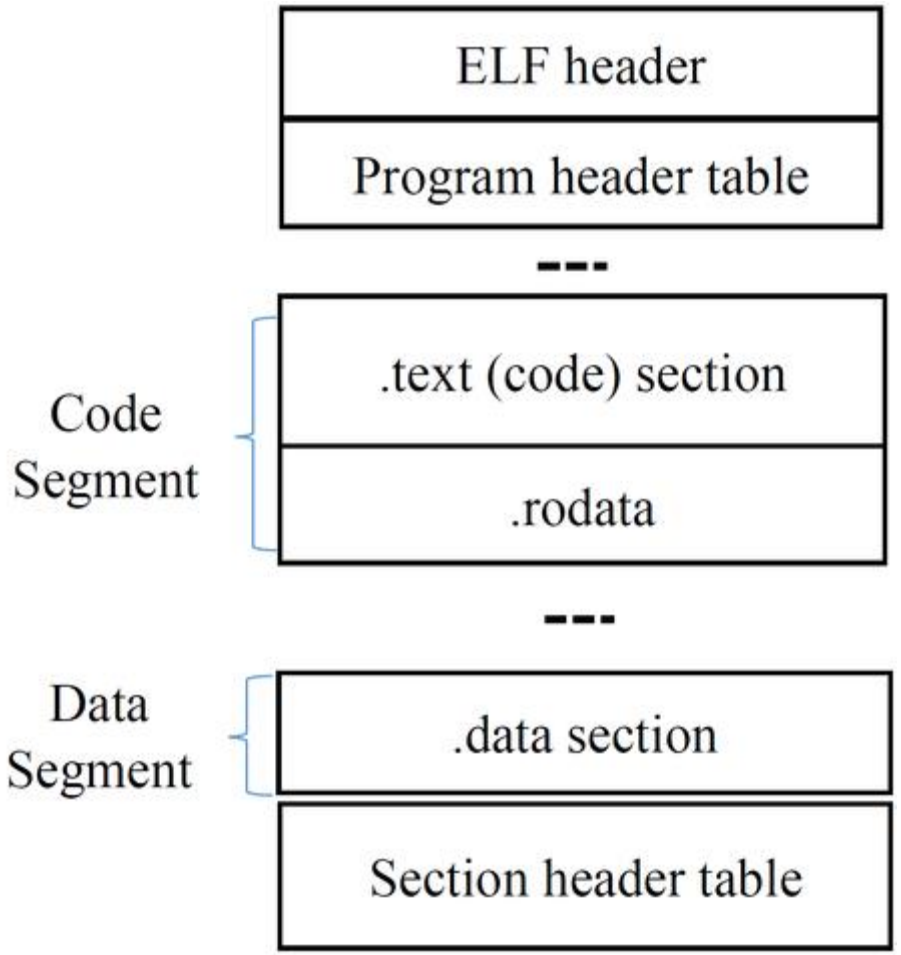
Supports:

- Object files
- Executables
- Shared libraries

# ELF File Structure Overview

ELF file contains:

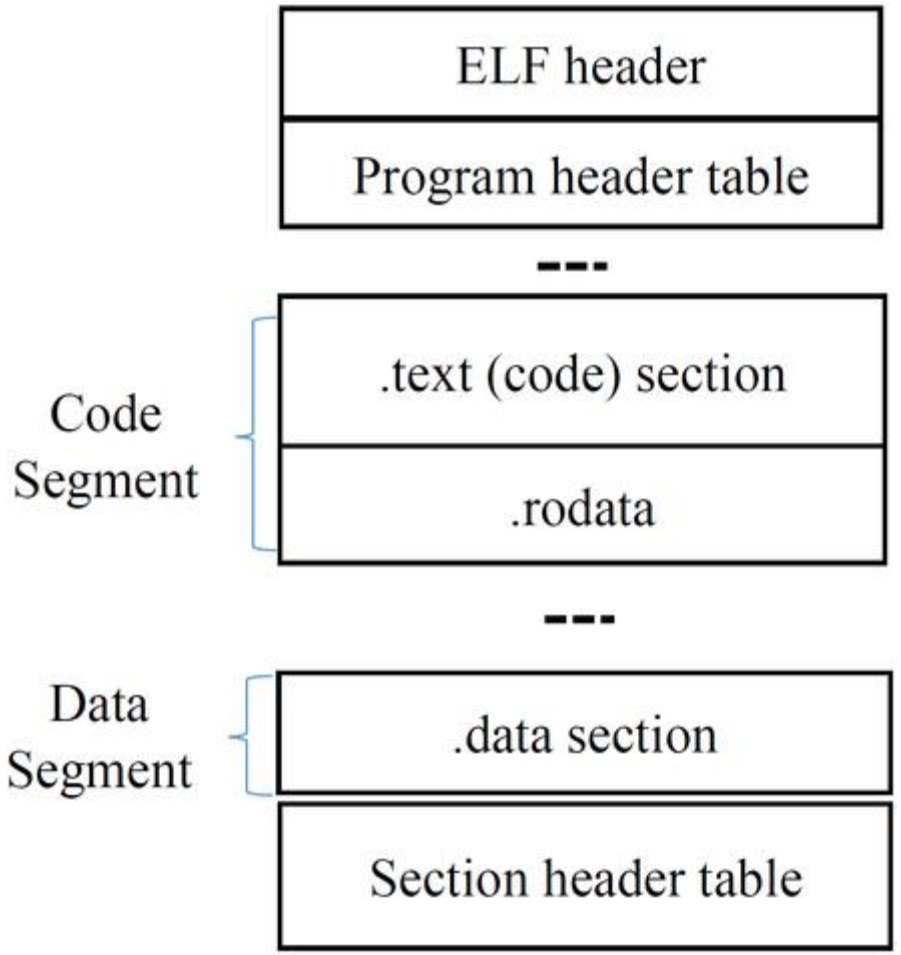
- ELF Header
- Program Header Table
- Section Header Table
- Sections



# ELF Header

Contains:

- Magic number
- Architecture
- Entry point
- Offsets to header tables
- File type (object, executable, shared)



# ELF Header Diagram

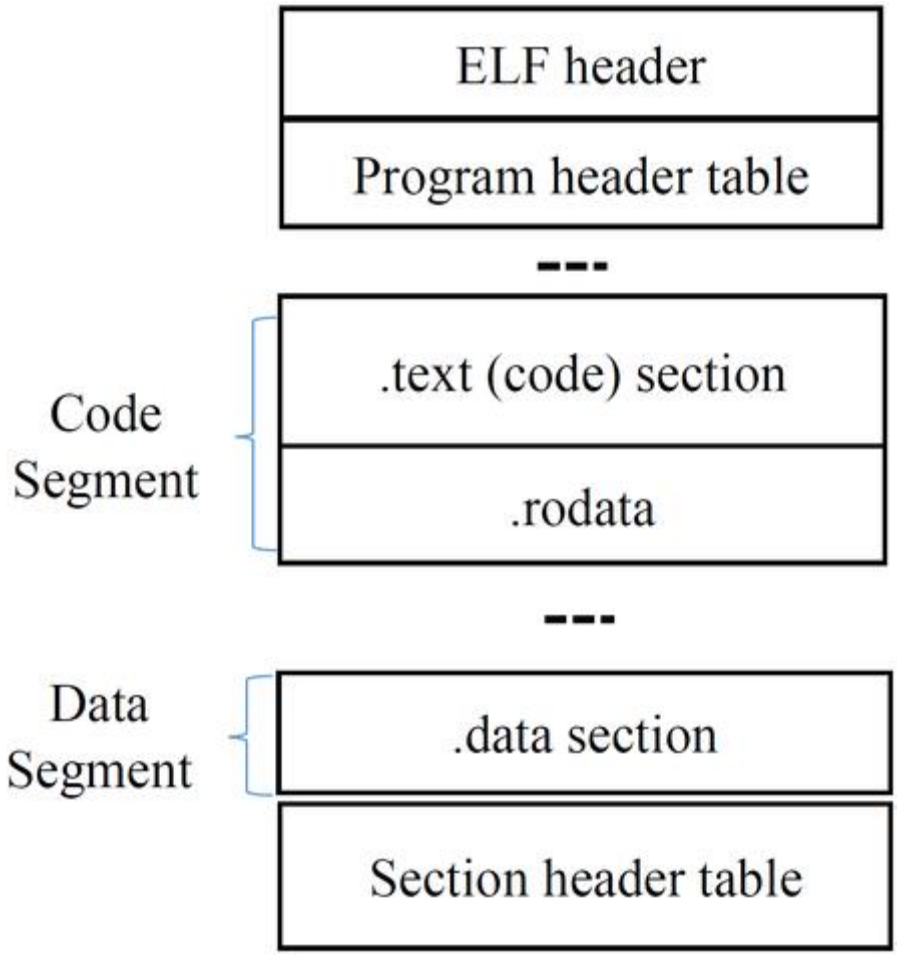
```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x4013e2
  Start of program headers:          64 (bytes into file)
  Start of section headers:          25376 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         9
  Size of section headers:           64 (bytes)
  Number of section headers:         28
  Section header string table index: 27
```

# Program Header Table

Used by Loader

Describes:

- Segments to load
- Virtual addresses
- Memory size
- Permissions

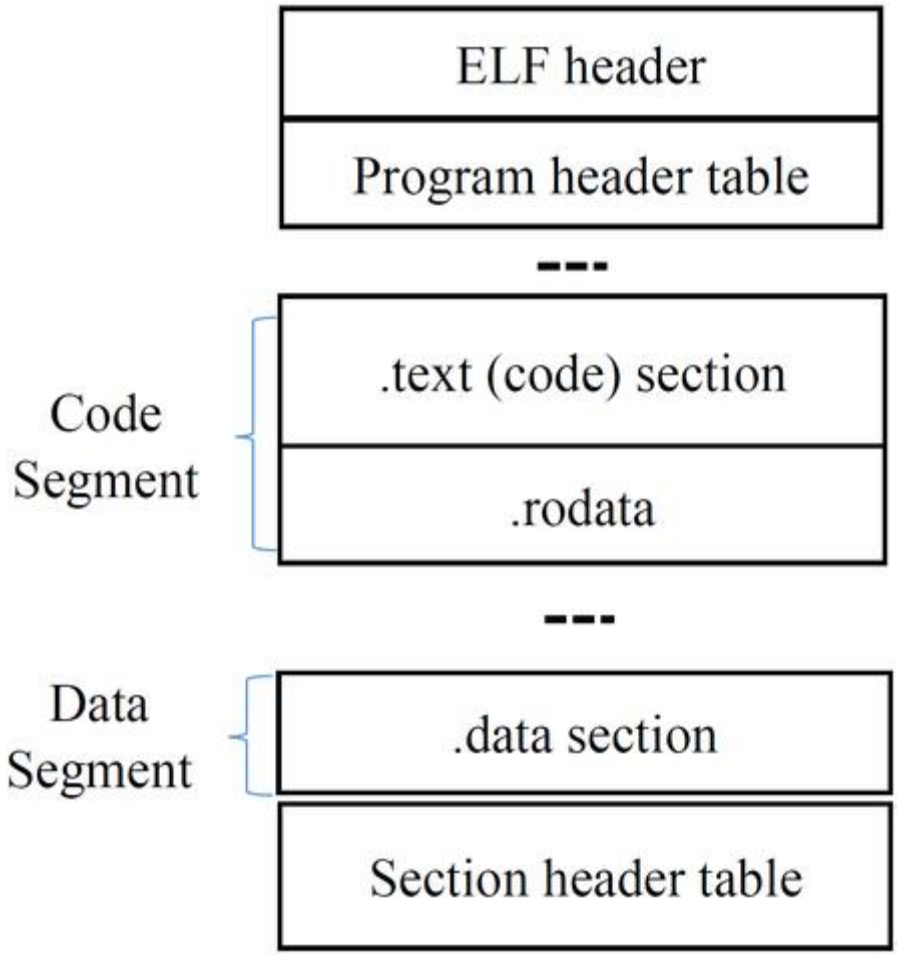


# Section Header Table

Used by Linker.

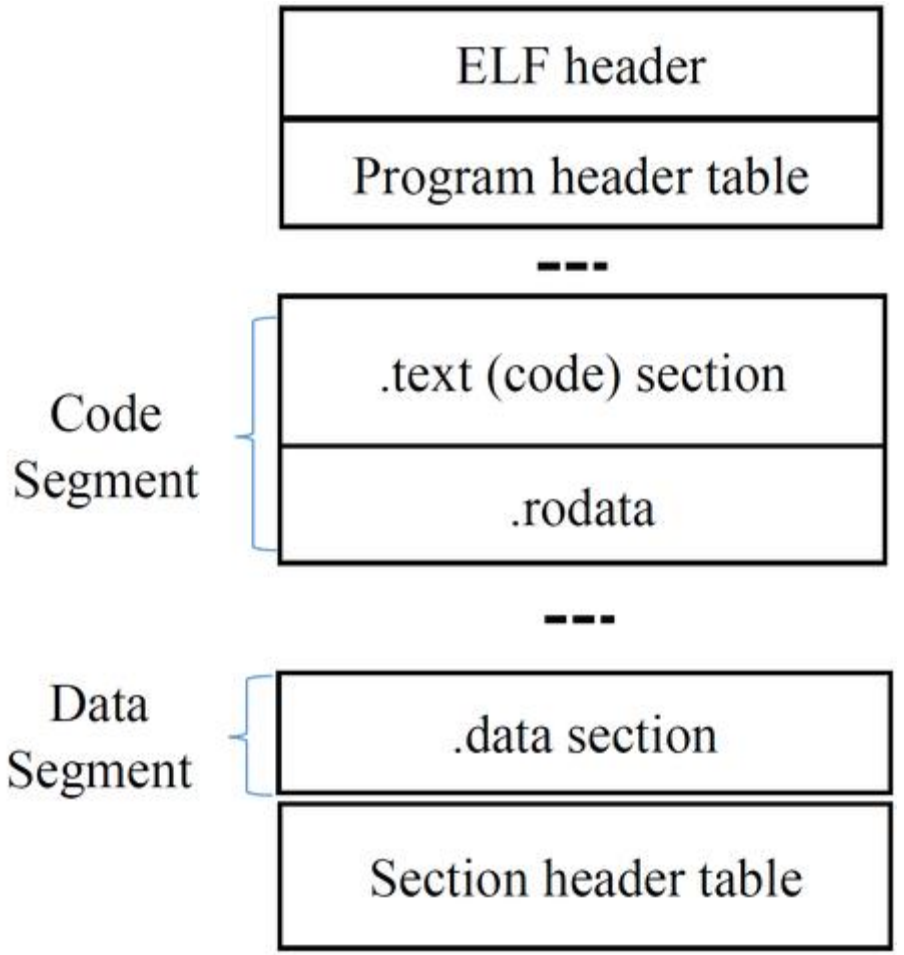
Describes:

- Section name
- Type
- Offset
- Size
- Flags



# ELF Sections

- text : code
- .data : initialized data
- .bss : uninitialized data
- .symtab: symbol table
- .strtab: string table
- .rel / .rela: relocation entries

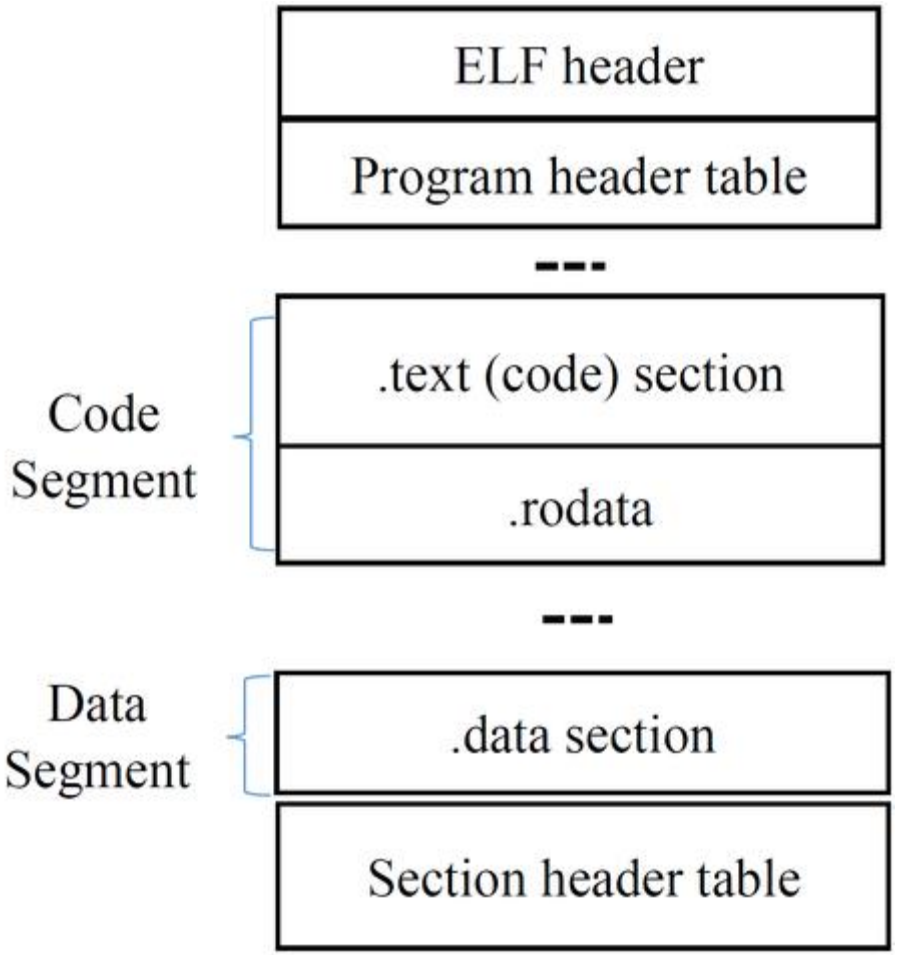


# Symbol Table (.symtab)

Contains:

- Symbol name
- Address
- Size
- Type
- Binding (local/global)

Used during linking.



# Relocation Sections

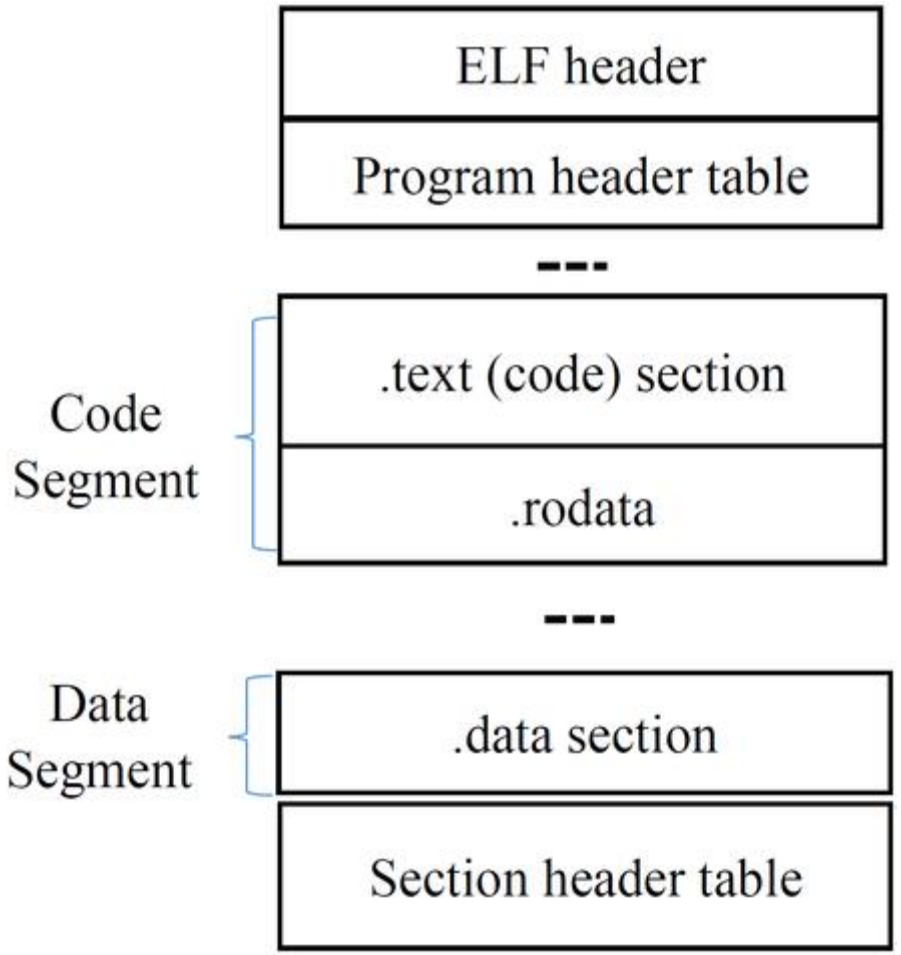
Contains:

- Offset to modify
- Symbol reference
- Relocation type

Used by linker and dynamic linker.

Dynamic ELF sections:

- .dynsym
- .dynstr
- .got
- .plt
- .dynamic



# Relocation Sections

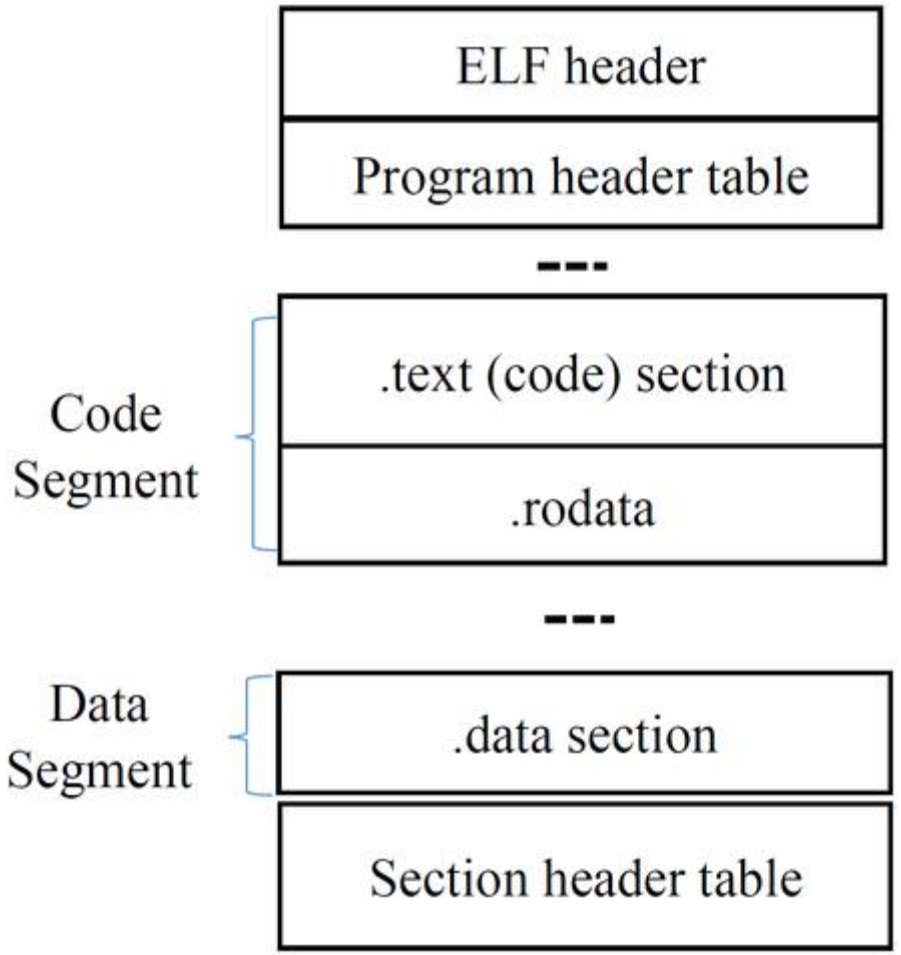
**GOT (Global Offset Table) .got**

Stores addresses of:

- External functions
- Shared library symbols

**PLT (Procedure Linkage Table) .plt**

- Used for lazy binding
- Indirect jump mechanism
- Updated after first function call



# Linker vs Loader

**Linker:**

- Combines modules
- Produces executable

**Loader:**

- Loads executable into memory
- Starts execution

# Summary

- Linker resolves cross-module dependencies
- Two-pass design simplifies resolution
- Static vs Dynamic linking serve different needs
- ELF defines modern executable structure
- Modern systems rely heavily on dynamic linking